

Week 01: Comparative Study of Programming Languages (Java, C, C++, Python, JavaScript)

Student: Trivikram Roll No: 25EU02067 • Lab Record Verified

1. OBJECTIVE & SCOPE

To perform a detailed comparative analysis of five foundational programming languages: **Java**, **C**, **C++**, **Python**, and **JavaScript**. This study evaluates programming paradigms, memory architecture, translation mechanisms, type systems, performance trade-offs, and industrial application domains.

2. COMPREHENSIVE COMPARATIVE TABLE

PARAMETER / FEATURE	JAVA	JAVA	C	C	C++	C++	PY
Programming Paradigm	Object-Oriented (Strict Class-based), Concurrent		Procedural, Structured, Imperative		Multi-paradigm (Procedural, OOP, Generic, Functional)		Mu (OO Fun Imp Scri
Platform Independence	Yes (Platform Independent via Bytecode & JVM - "WORA")		No (Platform dependent; source must be recompiled for each OS/CPU)		No (Platform dependent; compiled directly to native machine code)		Yes inde byt inte
Translation / Execution	Compiled to Bytecode (.class), then executed via JVM + JIT Compiler		Compiled directly to native binary/machine code (.exe / .o)		Compiled directly to native binary/machine code		Inte (Co byt exe CPy
Memory Management	Automatic Garbage		Manual (malloc),		Manual (new, delete), RAII &		Aut Gar

PARAMETER / FEATURE	JAVA	JAVA	C	C	C++	C++	PYTHON
	Collection (GC); No manual free/delete		`calloc()`, `free()`)		Smart Pointers (`std::unique_ptr`)		Col (Re Cou Cyc
Type System	Statically Typed & Strongly Typed		Statically Typed & Weakly Typed		Statically Typed & Strongly Typed		Dyn Typ Stro
Pointers & Memory Safety	Pointers supported internally; Explicit pointer arithmetic restricted for security		Full explicit pointer support and raw pointer arithmetic allowed		Full raw pointers, pointer arithmetic, and reference types		No poi vari obj refe
Concurrency & Threading	Built-in Thread class, Runnable, synchronization primitives, Virtual Threads		POSIX Threads (`pthread.h`), OS-level threading libraries		Standard library `std::thread`, `std::async`, mutexes		Thre libr by C Inte / Gl mu asy
Syntax Complexity	Moderate, strict, verbose, structured		Low complexity, concise, lower- level primitives		High complexity (Templates, multiple inheritance, operator overloading)		Mir ind bas rea
Performance & Speed	Very Fast (JIT optimization approaches native speeds)		Fastest (Direct hardware access, zero runtime overhead)		Near fastest (Zero-cost abstractions, inline assembly)		Slov (Int ove offl inte to C
Primary Industry Applications	Enterprise Systems, Android, Cloud Microservices		Operating Systems (Linux/Unix), Embedded		Game Engines (Unreal), High- Frequency Trading, Graphic Systems		Arti Inte Ma Lea

PARAMETER / FEATURE	JAVA	JAVA	C	C	C++	C++	PY
		(Spring), FinTech		Systems, Drivers, IoT			Scie Aut

3. SYNTAX & IMPLEMENTATION COMPARISON ("HELLO WORLD" & VARIABLES)

A quick comparative overview of how each language defines entry points, variables, and output:

Java (Strict OOP)

Java

```
public class Demo {
    public static void main(String[] arg
        String name = "Trivikram";
        System.out.println("Hello, " + n
    }
}
```

C (Procedural)

C

```
#include <stdio.h>

int main() {
    char name[] = "Trivikram";
    printf("Hello, %s\n", name);
    return 0;
}
```

C++ (Multi-paradigm)

C++

```
#include <iostream>
#include <string>

int main() {
    std::string name = "Trivikram";
    std::cout << "Hello, " << name << "\
    return 0;
}
```

Python (Dynamic)

Python

```
name = "Trivikram"
print(f"Hello, {name}")
```

JavaScript (Event-Driven)

JavaScript

```
const name = "Trivikram";
console.log(`Hello, ${name}`);
```

4. KEY OBSERVATIONS FOR JAVA PROGRAMMING

Why Java for Enterprise & AI Systems?

- **Write Once, Run Anywhere (WORA):** Java source files compile to bytecode (`.class`) executed uniformly across any OS with a compatible Java Virtual Machine (JVM).
- **Robust Memory Protection:** Disallowing explicit pointer arithmetic eliminates memory corruption, buffer overflows, and segmentation faults prevalent in C/C++.

- **Enterprise Concurrency:** Standard multi-threading and modern Virtual Threads (Project Loom) deliver exceptional throughput for large distributed web services.